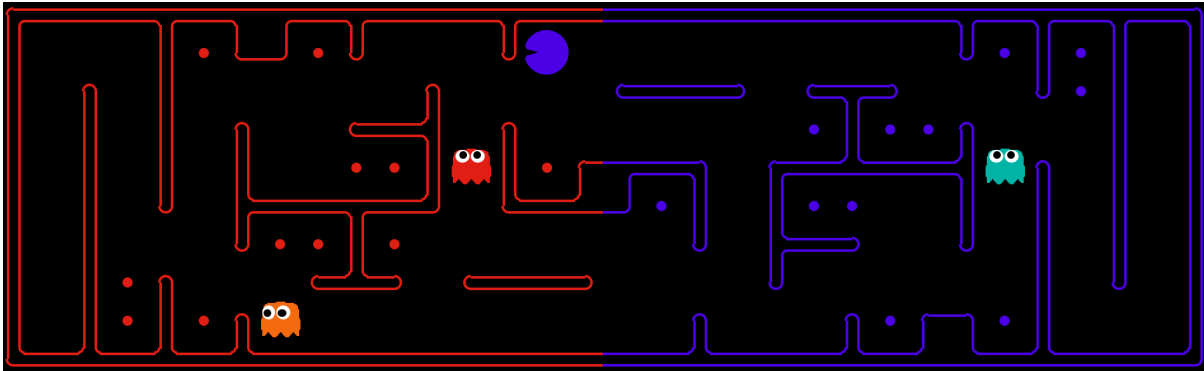


Assignment 2:

Pacman - Capture the Flag



In this assignment we will develop an AI controller for a multiplayer variant of Pacman. In this game two teams, red and blue, compete with each other to capture as many dots as possible from the opposite side of the map.

Originally developed at [UC Berkeley](https://www.eecs.berkeley.edu/~cs188/), this setup presents us with a challenging planning domain and an opportunity to solve a fun problem. At the end of the semester we will hold a contest to find out who developed the most effective controller. Glory and a fancy certificate await the top three students!

Part 1: Installation

Piglet PDDL Solver

In this assignment you will use the piglet library to solve PDDL problems. This requires installing piglet to the python environment:

1. Go to piglet-public folder (If you use a virtual environment, activate the virtual env first. You could use your flatland virtual environment as well.)
2. `git fetch`
3. `git checkout pddl_solver` //if successful, you should find a `pddl_solver.py` under `lib_piglet/utils/` folder
4. `python setup.py install`

Check the **myTeam.py** starter implementation to see how to use the newly installed library. The competition server will have the same piglet installed in the environment as a PDDL solver.

Update Pacman Code

Make sure you have the latest version of the Pacman CTF game environment:

1. Go to the pacman-public folder (you cloned the repo in week 1)
2. git fetch
3. git reset --hard //in case any changes you made locally
4. git pull
5. If your code is newest, you should see staffTeam.py and berkeleyTeam.py in the repo.

All codes for the game environment are now found in the **pacman** folder.

Part 2: Game Rules

We can characterise Pacman CTF as follows:

- **Multi-agent:** Two agents need to work together against an opposing team.
- **Discrete:** The environment is a grid maze and time is discretised into unit time steps.
- **Dynamic:** The environment changes as food is consumed.
- **Partially observable:** Each agent has a limited sensing range.
- **Sequential:** Past decisions affect future actions.
- **Deterministic:** The current state depends only on the teams and their actions.
- **Offline:** The world stops while an agent deliberates.
- **Known:** All the rules of the game are available a priori.

Environment

The game map is divided into two halves: red (left) and blue (right). When on the red side, a red agent is a ghost. When crossing into enemy territory, the agent becomes a Pacman. Red agents must defend the red food while trying to eat the blue food.

Scoring

When a Pacman agent eats a food dot, that dot is removed from the board and placed in a virtual backpack. When the agent returns to their own side of the board, they automatically "deposit" the contents of their backpack, earning one point per dot. The Red team scores positive points, while the Blue team scores negative points.

Watch out! If an agent is caught by a ghost, before reaching their own side of the board, all dots in their backpack will be deposited back to their original positions on the board. The agent also returns (as a ghost) to its original starting position. No points are scored in this case. No points are awarded to the opposing team either (for catching Pacman).

Power Capsules

A small number of power capsules are scattered throughout the maze. These can be eaten by a Pacman agent. When this happens ghosts become “scared” and are susceptible to being caught by the powered up Pacman. This ghost effect lasts for the next 40 timesteps, or until caught, whichever comes sooner.

Observations

Agents can only observe an opponent's configuration (position and direction) if they or their teammate are within 5 squares (Manhattan distance). In addition, an agent always gets a noisy distance reading for each agent on the board, which can be used to reason about unobserved opponents. The reading has an error of ± 6 steps from the true distance. A general direction (toward the opponent) is not specified.

Winning

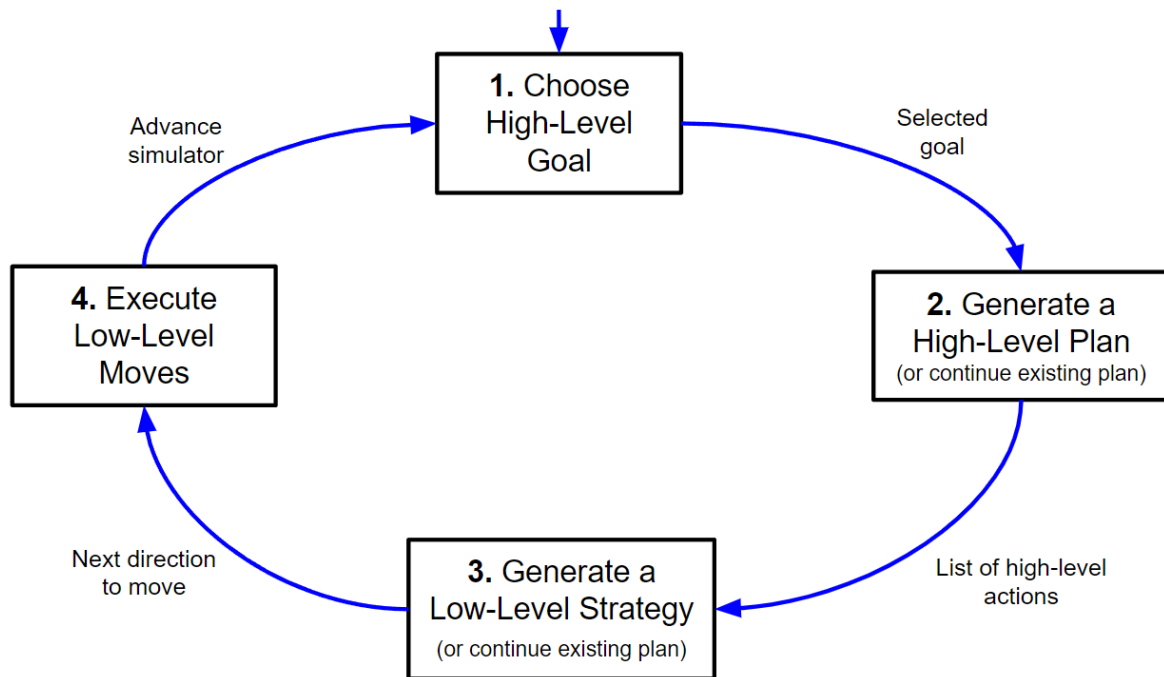
A game ends when one team returns all but two of the opponents' dots. Games are also limited to 300 time steps (i.e., 300 moves for each of the four agents). This equates to 1200 total actions, seen in the visualiser. If this move limit is reached, whichever team has returned the most food wins. If the score is zero (i.e., tied) this is recorded as a tie game.

Part 3: Task & Student Workflow

At every time step the simulator will present you with the current state of the game board. Your task will be to reason over this current state and decide how your agents should react.

This will require you to think about the planning process in several different ways: at the high-level, where you decide what strategy your team will pursue; at the low-level, where you transform that strategy into a concrete sequence of low-level actions; and you will need to decide when to replan your agents, in response to changing circumstances.

We illustrate an example of the *student workflow* in the diagram below.



Your starting point for modifying this workflow is the function **chooseAction** in file **myTeam.py**. This function is called by the simulator at every time step and for each agent. The simulator provides the agent with a description of the game board. At the end of the function you must return a concrete action for the agent to execute in the next timestep.

How to decide which action the agent should take is entirely up to you. The starting code provides a basic template for the decision-making process and some simple, though not very effective, strategies. You will need to modify this code to create a winning AI.

The next sections give further details about the decision-making process. For more details regarding the Pacman CTF code, the **myTeam.py** implementation, and other general tips, refer to the additional document “**Assignment 2 Code Documentation.pdf**”!

3.1. Choose a High Level Goal

What is the high-level strategy that agents should pursue? For example, they could visit the opposing side and try to score points, or they could stay home, and defend their score. Each agent chooses its own high level goal, but that decision may be influenced by the current state of the board, the current score, and the strategy and position of the teammate agent.

Your starting point here is the **getGoals** function in the file **myTeam.py**. Given a PDDL description of the current state, this function returns positive and negative PDDL state descriptions that together describe a high-level goal. You can create multiple different goals and then choose between them based on the information in the current state.

```
def getGoals(self, objects: List[Tuple], initState: List[Tuple]):
    # Check a list of goal functions from high priority to low priority if the goal is applicable
    # Return the pddl goal states for selected goal function
    if ("winning_gt5",) in initState:
        return self.goalDefWinning(objects, initState)
    else:
        return self.goalScoring(objects, initState)
```

3.2. Generate a High Level Plan

To achieve your high-level goals you will need to compute a corresponding sequence of high-level actions (i.e., a plan). Given a PDDL description of the initial (i.e., the current) states and the goal states, the function **getHighLevelPlan** computes such a plan. You do not need to modify this function. Just call it as necessary.

But what if the agent already has a high-level plan, computed in a previous iteration? It's up to you to decide how to proceed. The agent could retain the existing plan or it could change, and replan anew from the current state. For example, if the pre-conditions for the next action are no longer satisfied, the agent will certainly need to generate a new high level plan. The agent may also decide to revise its current plan (and possibly current goal!) if its execution produces an unexpected result (e.g., being caught by a ghost or a powered-up Pacman).

The staff implementation checks if we can reuse existing plan:

```
# Collect objects and init states from gameState
objects, initState = self.get_pddl_state(gameState)
positiveGoal, negativeGoal = self.getGoals(objects, initState)

# Check if we can stick to current plan
if not self.stateSatisfyCurrentPlan(initState, positiveGoal, negativeGoal):
    # Cannot stick to current plan, prepare goals and replan
    print("Agent:", self.index, "compute plan:")
    print("\tOBJ:" + str(objects), "\tINIT:" + str(initState), "\tPOSITIVE_GOAL:" + str(positiveGoal))
    self.highLevelPlan: List[Tuple[Action, pddl_state]] = self.getHighLevelPlan(objects,
    self.currentActionIndex = 0
    self.lowLevelPlan = [] # reset low level plan
    print("\tPLAN:", self.highLevelPlan)
if len(self.highLevelPlan) == 0:
    raise Exception("Solver returned empty plan, you need to think how you handle this situation")

# Get next action from the plan
highLevelAction = self.highLevelPlan[self.currentActionIndex][0].name
```

Note: Only a limited set of predicates are included in the basic PDDL state description (see the file **myTeam.pddl**) and not all of them appear in the given actions. You can create a richer model, and compute more diverse plans, by (1) refining existing actions or creating new ones that use more of the available predicates; (2) enhancing the PDDL state description with additional information from the current game state (i.e., add more predicates). We provide useful programmatic facilities for

this purpose. See the **get_pddl_state** function for examples of how to collect information from the simulator environment (**gameState**).

3.3. Generate a Low Level Plan

High level plans tell the agents what to do, but in broad strokes. For example, if the goal is to score points, a necessary action is to move to the opposite side of the board. But how should the agent get there? Once on the opposing side, the next high level action might be to eat a food dot. But which one? Do I want to move left, right, up, or down? Low level planning provides answers to these questions.

The function **getLowLevelPlan** is your starting point here. Its inputs are a high-level action and a description of the current game state. The function returns a sequence of move actions that the agent needs to take to achieve the high-level effects of the input action.

You are free to implement this function using any method you choose. For example:

- **Heuristic Search Strategy:** you can develop your own custom strategies by implementing a new expander class for your own customised state-space representation.
- **Learning Based Strategy:** we provide a very basic example of this approach. You can make your own implementation or improve it by introducing more features to the model. Then you will train the new model to obtain a set of good weights. You should carefully consider, for each high level action, how you will reward or penalise the agent for its performance. You may need to keep adjusting your implementation, observe how your agent behaves, and try something new to overcome the drawbacks you observed.

But what if the agent already has a low-level plan, computed during a previous iteration? Again, it's up to you to decide how to proceed. The agent could continue following the existing low-level plan or it might decide to revisit its decisions on account of new information provided by the game state. For example, if Pacman is heading towards a food dot, but observes a ghost up ahead, continuing toward the dot may be a bad idea.

The staff implementation reuses low level plans:

```
#-----Low Level Plan Section-----
# Get the low level plan using Q learning, and return a low level action at last.
# A low level action is defined in Directions, whihc include {"North", "South", "

if not self.posSatisfyLowLevelPlan(gameState):
    self.lowLevelPlan = self.getLowLevelPlanQL(gameState, highLevelAction) #Gener
    # you can replace the getLowLevelPlanQL with getLowLevelPlanHS and implement
    self.lowLevelActionIndex = 0
lowLevelAction = self.lowLevelPlan[self.lowLevelActionIndex][0]
self.lowLevelActionIndex+=1
print("\tAgent:", self.index, lowLevelAction)
return lowLevelAction
```

Remember! There are many different ways to achieve a high-level action. You may also wish to consider whether your two agents coordinate their low-level actions or act independently of one another.

3.4. Execute Moves

Your low-level strategy has generated a sequence of directions for the agent to move in. The first move in this plan is given to the simulator for execution. Each of your two agents, and the two opposing agents, move according to their own instructions and the timestep is advanced by one. All of this happens after the completion of the **chooseAction** function.

Based on the new state of the game board, you may need to re-evaluate your goals and plans. This workflow is cyclic and continues until the game is over.

Remember: You can (and should!) compare your implementation (**myTeam.py**) against the existing baselines (**staffTeam.py** or **berkeleyTeam.py**). You can find these implementations in the **pacman** folder. The baselines are useful to check that your implementation works and to measure your progress.

Part 4: Competition Setup

We will have a competition to see who has the strongest pacman AI. You will be able to submit your implementation to a contest server where they will be run against each other.

Upon submission to the server your agent will be evaluated against a staff baseline implementation (**staffTeam.py**). Your agent will be evaluated for 49 games on 7 different maps. Within each game there will be a time limit and the agent that has succeeded in eating the most food during that time will win that game. To win the match with staff baseline, your team must win convincingly: 28 out of 49 games.

By winning the **staffTeam.py**, you will get a pass for Criteria 1 competition score (see more details in marking rubric), have a position on the leaderboard, and your agent will be evaluated against all other participants on the leaderboard to get additional victory points.

With each opponent on the leaderboard (excluding the staff baseline implementation), your agent will be evaluated on the same set of problems:

- In the event of a convincing win (win at least 28 out of 49 games), you will get 3 victory points.
- In the event of a tie (win less than 28 games, but lose less than 28 games as well, considering tie games neither win nor lost.), you will get 0.5 victory points.
- In the event of a loss (lose 28 games or more), you will get 0 victory points.

If you win the staff baseline implementation, your marks for Criteria 1 competition will be decided by $(\text{Your total victory points} / \text{All available victory points})\%$, where the all available victory points is $3 \times (\text{number of all participants} - 1)$.

You are expected to **clearly win against the staff baseline implementation**, otherwise you will not proceed to the next stage of competing with entries from other students. In this case your grade for Criteria 1 of the marking rubric will be a Fail.

The server will keep a record of your matches against every opponent on the leaderboard. When someone implements a new personal-best record, your score against that student will be updated as well.

More details regarding the competition setup will be provided in a supplementary document ***“Assignment 2 Contest Submission Instruction.pdf”***.

Part 5: Report

Together with your implementation source code you will be required to submit a report that describes your approach. You will need to write a description of your strategy and give a justification for algorithmic choices. Make sure that your report is as detailed and complete as possible, including appropriate references. **There will be a 15 page limit on your report (not including references).** Minimum font size of 12 pt (Times New Roman or Arial). *Markers will stop reading after the page limit in which case your report may be marked as incomplete.*

When describing your model you should explain not only what it does but also analyse its strengths and weaknesses (e.g., complexity, guarantees etc). You should also discuss how your code implements that model: for example, important data structures, necessary optimisations and parameter choices. Justify your design and implementation choices with discussion and, where appropriate, with experiments.

You can ask questions about the assignment and you can discuss the merits of different design choices (e.g., on Ed). **However** all submitted work must be **completely** your own. Do not copy implementations you happen to find online. Do not ask your friends or colleagues for their implementations. You will need to work independently and take reasonable steps to make sure others' don't copy or misuse your work. For more information please see <https://www.monash.edu/students/study-support/academic-integrity>

PLAGIARISM IS NOT ACCEPTABLE.
YOU WILL FAIL THE ASSIGNMENT AND POSSIBLY THE ENTIRE UNIT.

Part 6: Marking Rubric

The rubric specifies the marking criteria for your implementation and report. To avoid disappointment, pay close attention to the requirements. The weight of the implementation is 66.6% while the weight of the report is 33.3%.

Marking Rubric - Implementation (100/150 marks)

Criteria	N 0%-49%	P 50%-59%	C 60%-69%	D 70%-79%	HD 80%-100%
1. Competition (50 marks)	Loses to the staff baseline implementation	Outranks staff baseline implementation.	Get 25% ~ 49% of all available victory points.	Get 50% ~ 74% of all available victory points.	Get 75% ~ 100% of all available victory points.
2. Agent strategy implementation (50 marks)	Minor updates to example PDDL implementation in the code base (for example, only changing parameter values)	Addresses some drawbacks in existing implementation. Improves the model and high level planner by: (1) Introducing new actions to the PDDL model, which uses more listed predicates or new predicates you introduced. Or (2) Design new goal functions.	Same as previous criteria plus: Implements alternative high level goal prioritisation scheme (agents switch goals based on new observations). Improves low level planner; e.g. more features, better reward functions, or uses heuristic search for low level planning.	Same as previous criteria plus: Customised low level plans for distinct high level actions; e.g. different low-level strategies for eat food (which?) vs. go to opposing side (where?).	Same as previous criteria plus: High level and low level decisions take into account the current action and strategy of the teammate agent. (Two agents should cooperate with each other, and share information with each other.) Or design your own model and agent workflow.

Marking Rubric - Report (50/150 marks)

Criteria	N 0%-49%	P 50%-59%	C 60%-69%	D 70%-79%	HD 80%-100%
Approach Description (15 marks)	Incomplete or insufficient description of the approach. Does not demonstrate sufficient understanding of the implemented approach and related algorithms. Little or no pseudo-code.	There is a description of the approach, but the approach is generally unclear. Little or no evidence to support the discussion. The discussion demonstrates a limited understanding of the implemented approach and related algorithms. Should describe and discuss how the new implementation improves upon the staff baseline, staffTeam.py	There is a description of the approach that can be mostly understood, except in respect of small details. Should provide some motivation for your implementation choices and link the agent strategy to lectures and tutorials. There is limited evidence and a limited argument justifying the choice of approach. The discussion demonstrates a baseline understanding of the implemented approach and related algorithms.	There is a description of the customised low level plans and actions that can be fully understood. There is a clear but somewhat flawed argument justifying the methodology. The approach is supported by evidence that is somewhat relevant. The discussion demonstrates good understanding of the implemented approach and related algorithms, but with minor issues.	There is a description of the approach that can be fully understood. The quality of the justification of the methodology is supported by strong evidence. The discussion demonstrates excellent understanding of the implemented approach and related algorithms. Provide clear and theoretical motivation for your implementation choices and directly link the agent strategy to lectures and tutorials.
Experiment Discussion (15 marks)	Provides little or no discussion of experimental work. Results are missing, incomplete, or unsupported by evidence. No	Provides a basic discussion of experimental results with limited evidence. Some numerical details may be included but are incomplete or lack context. Mentions efficiency of implementation in	Presents a sound discussion of experiments with supporting numerical details. Results are generally clear and partially analysed for efficiency. Some attempt to evaluate implementations	Provides a strong and well-supported discussion of experiments, comparing at least 2 implemented strategies/attempts. Numerical evidence is	Demonstrates a comprehensive and insightful discussion of experiments. Numerical results are detailed, accurate, and fully integrated into the analysis, comparing at least 3 implemented

	numerical details are presented, and efficiency or implementation performance is not addressed. Work shows major flaws or misunderstanding of the experimental process.	vague terms, without meaningful analysis or comparison. Discussion shows only minimal engagement with experiments performed.	against benchmarks or references is made, but analysis may be superficial or inconsistent. Shows adequate understanding of the experimental process.	clearly presented and used to evaluate efficiency of implementations. Comparative analysis against standard benchmarks and/or appropriate reference algorithms is evident and accurate. Discussion demonstrates critical engagement, though some aspects may lack full depth.	strategies/attempts. Efficiency of implementations is rigorously assessed on standard benchmarks and compared effectively to appropriate reference algorithms. Evidence is consistently used to support claims, and discussion reflects exceptional depth, precision, and critical evaluation.
Reflections (10 marks)	Lacks critical thinking and original insights. Discussion of the strengths and weaknesses of the implemented approach are absent or irrelevant.	Limited critical thinking and few original insights. Conclusions are vague or poorly articulated, but touch upon some strengths and weaknesses of the implemented approach.	Demonstrates basic critical thinking with limited insights. Provides some conclusions, but misses key implications relevant to the implemented approach.	Shows good critical thinking with some reflections on the strengths and weaknesses of implemented approaches. Provides clear discussion of implications, but misses some key points.	Demonstrates strong critical thinking and provides insightful reflections on the strengths and weaknesses of implemented approaches. Clearly articulates the implications of the findings.
Communication skills (10 marks)	Hard to follow with no clear narrative. Inadequate or no separation of discussion text into coherent sections.	The writing has a tenuously logical narrative. Some attempt at the expected structural elements (e.g. Intro, conclusion). Writing is not accurate or articulate most of the time.	The text has a clear logical narrative and expected structural elements (e.g. intro, conclusion). Writing is not accurate or articulate most of the time.	The writing is well composed and has a clear and logical narrative and is well structured. Writing is generally accurate and articulate.	The writing is very well composed and has a very clear and logically-formed narrative as a whole. Writing is accurate and articulate.

	Writing is not accurate or articulate. Inadequate supporting materials. Inadequate or missing referencing.	The document has few supporting materials (tables, images, pseudo-code) . The student has attempted to undertake citing and referencing with frequent errors.	There are some supporting materials (tables, images, pseudo-code) but not well integrated with the rest of the text. The student follows the requirements for citing and referencing, with some notable errors.	The document has appropriate supporting materials that are well integrated with the rest of the text. The student follows the requirements for citing and referencing, with some minor errors.	The document is expertly structured in the style of a scientific report, including appropriate supporting materials that clearly improve the quality of associated discussion. The student follows the requirements for citing and referencing.
--	---	---	---	--	---

Part 7: Submission Guide

Submission to Moodle:

The following submission items are required:

1. Your implementation source codes, in a single directory called "src" (you can copy everything in the pacman folder to "src"). Zip the codes directory with file name last_name_student_id_pacman.zip. (For example, Chen_123456_pacman.zip)
2. The report describing your approaches as a single pdf file. Name the pdf as last_name_student_id_report_pacman.pdf. (For example, Chen_123456_report_pacman.pdf)

Submission to Contest Server:

You must make a successful submission on the contest server as part of your mark. Submit early and submit often. Don't wait for the last day, when you might be subject to circumstances out of your control (like server outages & queues!).

Submission deadline : 11:55 pm Monday 3rd November, 2025 (week 14)